

Mari: Responsive Wireless Communication for Low-Power Large-Scale Robot Swarms

Geovane Fedrecheski^a, Yinghao Gao^b, Filip Maksimovic^a, Alexandre Abadie^a,
Mališa Vučinić^a, Tengfei Chang^b, Thomas Watteyne

^a*Inria, France*

^b*HKUST(GZ), China*

Abstract

Connecting large swarms of low-power micro-robots remains an open challenge: existing communication systems either lack the scalability to connect hundreds of units, or fail to deliver the low latency required for responsive control. For example, while Wi-Fi and BLE are commonly used in swarm robotics, scaling beyond 50 robots remains challenging. We introduce the Micro-robot Access Radio Infrastructure (**Mari**), a new link layer and architecture that aims to connect thousands of robots. Our TSCH-based link layer runs on commodity BLE radios, providing responsive control of up to 102 nodes using a single gateway. To scale beyond a single gateway, **Mari** supports multiple non-coordinated gateways with fast node-controlled handovers. We further provide a lightweight infrastructure for real-time remote control and monitoring. We implement **Mari** on the popular nRF52840 and nRF5340 platforms, and extensively evaluate it in large-scale testbeds. In experiments with 100 nodes connected to a single gateway, **Mari** achieves a packet delivery ratio above 97%, round-trip latency of 269 ms, and full association time of 3.5 s under a join storm. When moving 100 nodes together between two gateways, we obtain an average handover duration of only 0.34 s. A single node handover time can take less than 50 ms. In a setup with 200 nodes and two gateways, we measure round-trip latency of 274 ms and a packet delivery ratio above 93%. We integrate **Mari** with the

Email address: geovane.fedrecheski@inria.fr (Geovane Fedrecheski)

DotBot testbed in an experimental deployment of several hundred robots, enabling remote firmware update of 600 robots in less than 4 min. We contribute **Mari** as an open-source platform, including firmware, libraries, and performance monitoring tools, enabling researchers to connect swarm systems from a handful of units up to hundreds.

Keywords: swarm robotics, networking, testbeds, TSCH, BLE

1. Introduction

Large-scale robotic swarms are the next frontier in distributed autonomous systems. Swarms of micro-robots [1] could perform inspection and damage repair in hard-to-reach places, such as engines, building interiors, and satellites in orbit. Their applications span from warehouse automation [2] to innovative user interfaces [3] and minimally invasive medicine [4]. Large swarms of small robots can also be useful to experiment with swarm behavior in more realistic scenarios before committing to full-scale robots, helping to close the simulation-reality gap that is common in swarm robotics.

Communication is a major challenge to the advance of swarm systems [5]. This includes not only peer-to-peer (P2P) communication among close-by robots, but also communication with “control towers” used to control and deploy a swarm. That is, even if robots have a robust P2P communication system, they still need some communication link with a central entity to receive missions. This includes the user interface and the user experience involved in human-swarm interactions, where users can want to dynamically change the behavior of a swarm dynamically [6]. Moreover, a global communication link with swarms is needed to perform operational tasks, such as uploading firmware [7] and collecting logs.

As a concrete use case, we are building the DotBot testbed, comprising 1,000 small wheeled robots designed for indoor environments, e.g. a building floor. The robots, called DotBots, are cheap and low-power. They feature a dual-core ARM Cortex-M33 processor and a BLE-compatible radio. The

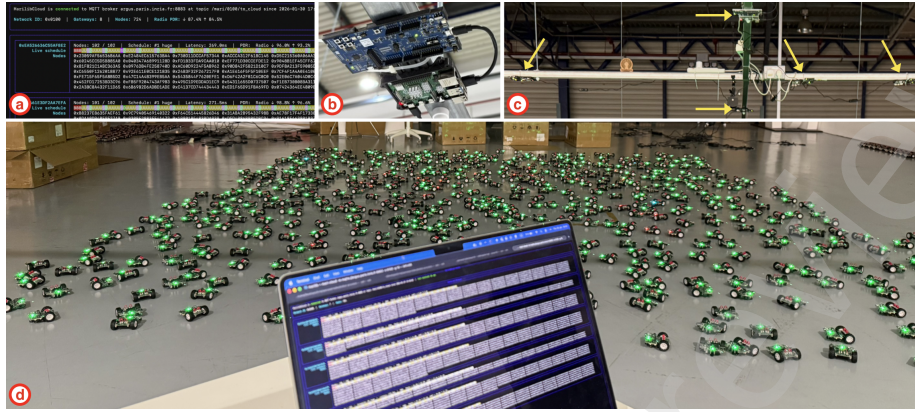


Figure 1: **Mari** allows communication with hundreds of robots using BLE radios. (a) Command-line dashboard showing real-time network statistics, including schedule usage, average end-to-end latency (270 ms) and overall PDR (above 85%). (b) A single **Mari** gateway, serving up to 102 nodes. (c) Gateways mounted above the robots. (d) Robotic swarm on the floor, with a laptop monitoring the real-time network status.

testbed allows researchers to monitor, control, and update firmware to the robots. Offering such functionality in a testbed of 1,000 robots poses a significant communication challenge.

Existing communication systems adopted in swarms are typically limited to a few dozen robots [8, 9, 10, 11, 12, 13], with few works reaching the scale of 50 robots or more [14, 15, 16]. Cellular systems, such as 5G, offer ultra large scale and node mobility, but are inappropriate for swarms of micro-robots due to high cost and power consumption. On the other hand, approaches leveraging low-power radios and Time-Synchronized Channel Hopping (TSCH) are well established [17, 18], but lack bandwidth for low-latency control and were designed for static nodes. A design with multiple synchronized TSCH gateways [2] addresses mobility, but its reliance on a replicated schedule hinders horizontal scalability.

This paper addresses the challenge of providing wireless connectivity for ultra-large low-power robot swarms. Our goal is to create a plug-and-play connectivity backbone to the swarm. It should enable creation of tools for remote

monitoring, interaction, and experimentation with thousands of micro-robots, ultimately unlocking an unprecedented scale for swarm experimentation.

We propose the Micro-robot Access Radio Infrastructure (**Mari**), comprising a new link layer and an accompanying infrastructure. Our TSCH-based link layer on top of a BLE PHY enables deployments at the scale of hundreds of robots, responsive control and fast over-the-air programming (OTAP), as well as multi-gateway fast handovers. The use of TSCH allows scaling the network while avoiding packet collisions and channel-dependent issues. While TSCH commonly uses schedule configurations that favor saving energy, we develop a custom schedule for **Mari** that maximizes radio usage. This ensures **Mari** is responsive for controlling 100 robots using a single gateway, and has enough throughput for OTAP updates.

To support deployments with thousands of nodes, **Mari** supports multiple non-coordinated gateways. Each gateway operates independently, while nodes implement handover logic locally. We adopt a semi-static TSCH schedule approach that enables sub 50 ms handovers. On the infrastructure side, we use MQTT to exchange messages between the user’s computer and the gateways. We provide a C implementation of the **Mari** link layer running on low-power commercial off-the-shelf (COTS) MCUs, and a Python library that enables users to easily interact with **Mari**-enabled swarms.

We evaluate **Mari** in real hardware using up to 200 nodes. We measure network formation time during a “join storm”, achieving 100 joined nodes in less than 4 seconds. When 100 nodes move together, simulating a flocking behavior, 50% of the nodes perform a handover in less than 0.312 seconds. **Mari** achieves average roundtrip latency figures from 45.7 ms up to 274 ms, with 10 and 200 connected nodes, respectively. In a test with up to 100 connected nodes, we achieve an average PDR above 97%. We also integrate **Mari** with the DotBot testbed in an experimental deployment of several hundred robots, enabling OTAP update of 600 robots in 4 minutes.

The contributions of this paper are four-fold:

- A wireless link layer demonstrated to responsively connect 724 nodes. **Mari** is, to the best of our knowledge, the first system to enable responsive control of mobile nodes at this scale using BLE radios.
- A TSCH-based architecture supporting multiple independent gateways and fast node-controlled handovers (< 50 ms for a single node, 0.34 s average for 100 moving nodes).
- A set of semi-static TSCH schedules balancing scalability, latency, throughput, and mobility. The design achieves average latencies of 45 ms (10 nodes) and 269 ms (100 nodes), and includes a fast slot allocation mechanism that enables the above handover performance.
- An open-source communication stack comprising firmware for low-power MCUs and a Python library for observing and interacting with **Mari** networks, enabling rapid experimentation and reproducibility within the research community.

2. Related Work

We review communication technologies used in swarm robotics, including solutions to tackle scalability, low-latency, and mobility.

Wi-Fi and BLE are prevalent in the field of swarm robotics due to widespread support. Wi-Fi offers low latency and high throughput, enabling real-time transmission of rich information such as video streams and environment mapping. Limitations of Wi-Fi for swarms of micro-robots include relatively high power consumption, in the order of 200 mA (comparable to a motor), and scalability, due to its contention-based medium access. Some of the largest Wi-Fi-based swarms in the literature achieve 22 [19], 50 [15], and 63 [16] nodes each. A solution with 50 robots [15] reported packet loss of 40-50% due to high node density.

BLE provides a low-power alternative, typically drawing around 10 mA during transmission, an order of magnitude less than typical robot motors. Al-

though its maximum throughput of 2 Mbps is not enough for transferring rich media in real time, it is sufficient for robot control. In terms of scalability, the BLE software stack typically achieves 5 [3, 10] to 10 [11] nodes connected to a single gateway. In the HERMITS swarm [3], connecting 70 robots required a total of 14 BLE gateways. Bluetooth mesh enables larger coverage areas [20], but its flooding-based mechanism makes it unsuitable for real-time control.

An alternative consists in leveraging 2.4 GHz radios, sometimes reusing BLE modulation, and implementing a tailored link layer for the needs of multi-robot control. This is the case of the Zooids robot swarm [8, 21], which achieved 32 robots with 3 radio gateways. Similarly, the CrazySwarm system [14] modified the communication layer of the CrazyFlie quadrotor platform to tolerate packet loss and support compression, enabling control for a swarm of 49 quadrotors. It used 3 gateways, built out of a 2.4 GHz radio on the nRF24LU1+ MCU.

To allow scaling up to 1024 robots, the Kilobot [22] uses infrared communication, allowing massive OTAP of all robots. Due to bandwidth constraints, this setup does not allow real-time control.

The IEEE 802.15.4 standard [23] offers a PHY that is appropriate for low-power devices such as wireless sensors and Internet of Things (IoT) devices. It is used by higher-layer protocols such as ZigBee [23] and Thread [24], which use static mesh topologies. The IEEE 802.15.4e [17] amendment introduces TSCH support to IEEE 802.15.4, improving its scalability, power consumption, and resilience to interference. It has been studied and deployed in both academic [25] and industrial [26] settings, demonstrating the effectiveness of the TSCH approach. The IETF 6TiSCH stack further standardizes the integration of TSCH in IP networks [18], but mainly targets static nodes. The maximum speed of 250 kbps found in IEEE 802.15.4 networks can be a limiting factor for real-time control at very large scale, i.e. thousands of micro-robots.

TBLE [27] implements TSCH mesh routing on top of BLE, achieving 20% lower latency when compared to IEEE 802.15.4e networks. Due to faster data rates of up to 2 Mbps, it achieves a slot duration of 4 ms, shorter than the 10 ms slots in IEEE 802.15.4e. TBLE does not address the challenge of mobility in

TSCH systems.

A common solution to improve scalability in wireless systems consists in adding more gateways. This can also help to increase the coverage area, enabling larger deployments. Cellular systems such as 5G are built for ultra-large scale and have mobility built-in, but consume considerable power and are expensive to deploy. DECT-NR+, a cheaper alternative focused on industrial sensing and control, still consumes hundreds of mW [28], making it unsuitable for power-constrained micro-robots. Wi-Fi supports handovers, while standard BLE does not. Existing low-power swarm robotic systems primarily use multiple gateways to tackle scalability [3, 8, 14, 12], leaving the mobility aspect less explored.

Standard TSCH allows for extended range via mesh network formation, but does not support mobile nodes. One critical issue is the time needed to leave one neighbor and join another, which can take several minutes [29]. Adapted scheduling algorithms have been proposed to reduce re-join time, at the cost of increased communication delay [30]. Centralized scheduling schemes have been shown to be more effective at handling mobility in TSCH systems [31].

Haxhibeqiri et al. propose a synchronized single-hop multi-gateway (SHMG) architecture using TSCH to connect a set of mobile robots [2]. The solution uses a dedicated Precise Time Protocol (PTP) server to keep gateways synchronized. This allows robots to roam around an area covered by multiple gateways. Since the gateways are synchronized to a precision of 1 ms, robots communicate with a single logical gateway, when in reality more than one gateway may be receiving the same message. A backend server provides message de-duplication. One limitation of the SHMG architecture is communication overhead, as the schedule is replicated for each gateway. To solve this, a Location-Aware Scheduling Algorithm with Rescheduling (LASA-R) has been proposed to build and maintain an efficient TSCH schedule for robots considering their location [32]. The SHMG architecture has been validated with up to 4 robots and 2 gateways. Although it is expected to scale to larger deployments, a fundamental limitation is the fact that all gateways operate as a single logical gateway, hindering horizontal scalability.

A Shared-Downstream Dedicated-Upstream (SD-DU) scheduling scheme has been proposed to improve the efficiency of the SHMG architecture [33]. In SD-DU, downstream traffic can be shared across a set of robots, while upstream traffic is dedicated to each robot. We adopt a similar approach in **Mari**.

The SHMG architecture differs from **Mari** since our system does not require gateway synchronization, increasing scalability as each gateway serves a different set of robots. Moreover, since robots themselves control the handover procedure, network infrastructure remains minimal and easy to deploy. Finally, **Mari** uses a semi-static slot allocation scheme, with minimal communication overhead for scheduling.

Ben Yaala et al. [34] show that unsynchronized co-located TSCH networks suffer from collision periods due to clock drift, and propose cooperative resynchronization among border routers to mitigate the effect. Their analysis assumes IEEE 802.15.4e parameters with 15 ms slots and 16 channels, where blackout periods can exceed 100 s. **Mari** operates with shorter slots (1.7 ms), a larger channel pool (37 BLE data channels), and single-hop communication, reducing both the duration and probability of such collisions. Results in Section 5 confirm that PDR stays above 85% even when using eight co-located **Mari** gateways.

Among the reviewed solutions, only 5G offers scalability for very large scale deployments, at the cost of considerable power consumption and high deployment cost. Although most reviewed technologies enable low-latency, few achieve it at scale. **Mari** is the first system to enable responsive control of hundreds of mobile robots using BLE radios. While inspired by multi-gateway solutions, **Mari** provides a novel approach that does not require gateway synchronization. Similarly, **Mari** tackles the challenge of mobility in TSCH systems by using centralized scheduling, adopting a semi-static slot allocation scheme with minimal communication overhead. Finally, **Mari** stands out by providing a real-world deployment with more than 700 nodes, demonstrating the feasibility of the approach. Next, we describe the design of **Mari** in detail.

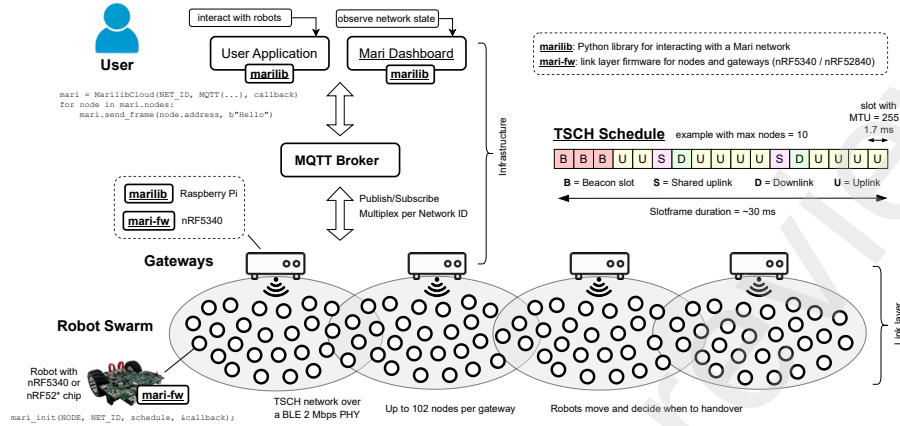


Figure 2: Architecture of Mari.

3. Design of Mari

We propose the Micro-robot Access Radio Infrastructure (**Mari**), a wireless link layer and infrastructure that enables responsive communication with large and dense swarms of low-power robots.

3.1. Requirements

The requirements for the design of **Mari** are:

- **Scalability (R1)**: future robot swarms will comprise hundreds to thousands of units, so **Mari** should natively support large scale deployments.
- **Responsiveness (R2)**: researchers should be able to observe and interactively control the robots, as well as to tune the network parameters, e.g. to allow remote control loops. Unlike traditional Wireless Sensor Networks (WSN) applications that optimize primarily for battery life, **Mari** must provide low end-to-end latency, in the order of 50 to 500 ms.
- **Throughput for OTAP (R3)**: reprogramming large scale swarms can be extremely time consuming if done manually, hindering rapid experimentation. **Mari** should provide sufficient throughput to support OTAP update of a full swarm in a matter of minutes.

- Fast association (**R4**): when a robot turns on or resets, or when a gateway suddenly becomes available, the whole swarm should rapidly associate, ideally in a few seconds.
- Seamless handovers (**R5**): handover events must be fast to minimize impacts on responsiveness. From a single robot perspective, the delay should be comparable to the average latency.
- Low-power radio (**R6**): the radio should consume a negligible amount of power when compared to the robot motors. Given that motors in small robots typically draw around 300 mA, and assuming a supercapacitor-based, low energy density design [35], a continuous radio consumption of 10 mA should allow operation for at least 2 h.

A final, non-functional requirement is that the system should be practical to deploy (**R7**), to facilitate adoption by the research community. That means it should be low-cost to allow scaling, ideally relying solely on COTS components. Infrastructure complexity should also be kept low, as to minimize deployment overhead.

3.2. Overview

As shown in Fig. 2, **Mari** has two main components: a wireless link layer and a communication infrastructure.

On the wireless side, we propose a novel link layer based on TSCH. We specify a TSCH schedule designed to balance several trade-offs: node count (**R1**), latency (**R2**), throughput (**R3**), and association time (**R4**, **R5**). We adopt BLE as the PHY layer due to its low-power operation (**R6**), data rate of up to 2 Mbps (**R2**, **R3**), and low-cost availability as a COTS item (**R7**). **Mari** supports multiple non-coordinated gateways, enabling horizontal scalability (**R1**). To enable node mobility, we incorporate a fast node-controlled handover logic (**R5**). We provide `mari-fw`¹, the link layer implementation in C for both node

¹<https://github.com/dotbots/mari>

and gateway, available under an open-source license.

On the infrastructure side, **Mari** uses a “star-of-stars” topology with multiple gateways and an MQTT broker used for message routing. Since gateways do not need to be coordinated among each other, deploying a **Mari** network is fast and requires minimal configuration (**R7**). To simplify access by users, we provide **marilib**², a Python tool that abstracts MQTT communication, parses **Mari** frames, and provides real-time network metrics.

When a robot wants to send data, it adds a frame to a queue, which is then transmitted in the robot’s assigned uplink slot. The gateway receives the frame and publishes it to the MQTT Broker, which is then received by the applications that are subscribed to the topic corresponding to that particular network. The reverse way is analogous, with the application sending a message downstream via MQTT. All gateways receive the message, and then filter it in or out based on the destination address.

Unlike prior TSCH-based systems designed for low-power and static sensor networks [36], **Mari** introduces a low-latency scheduler coupled with node-controlled handover and multiple non-coordinated gateways.

The following subsections describe the **Mari** components in detail. We present the frame format and its TSCH components, as well as the association and handover mechanisms. We also present the schedule we devised using a simulation-on-the-loop process to balance requirements **R1** to **R5**.

3.3. Frames

Fig. 3 shows the format of the five types of frames used in **Mari**: **Beacon**, **JoinRequest**, **JoinResponse**, **KeepAlive** and **Data**. All frames have fields to indicate the version and type, as well as a 2-byte network identifier.

Frames of type **JoinRequest** and **JoinResponse** are used during network formation. The **Data** frame carries regular data upstream or downstream. When there are no data to be sent upstream, nodes send a **KeepAlive** packet to let

²<https://github.com/dotbots/marilib>

1	1	2	8	8	1	1	128
version	type	net-id	asn	src	rem. cap.	sched.	bloom

Packet Types: Beacon = 1

1	1	2	8	8	0 - 235
version	type	net-id	dst	src	payload

Packet Types: JoinRequest = 2, JoinResponse = 4, KeepAlive = 8, Data = 16

Figure 3: Frame format in **Mari**.

the gateway know they are still in the network. These frames follow the same structure. They contain destination and source fields, and a payload of variable size. Since the header occupies 20 B, and BLE 2 Mbps supports frames of 255 B, a payload in **Mari** can have up to 235 B.

The **Beacon** frame is used to advertise the presence of a gateway. It contains information about the gateway, including the current Absolute Slot Number (ASN) and the gateway address (**src**). It also indicates the remaining capacity of the gateway (i.e. the number of nodes that can still associate to it), along with an identifier for the schedule currently in use. Finally, beacons carry a bloom filter field that allows a node to test whether it is joined to a particular gateway, as explained in Section 3.6.

3.4. Synchronization and Channel Hopping

The **Mari** link-layer implements Time-Synchronized Channel Hopping (TSCH). In TSCH, nodes communicate using assigned time-slots, selecting a different communication channel for each slot. Slots are synchronized with precision of a few microseconds.

A **Mari** node synchronizes its clock to a gateway when it receives a **Beacon** frame. Hardware differences make the node clock drift over time. As shown in Fig. 4, the receiver side knows that the sender begins a transmission exactly $TsTxOffset$ μ s after the beginning of the slot. By comparing the expected re-

ceive time with the actual time of starting to receive a frame, the node computes its own drift, and corrects it for the next slot. In case the drift is larger than *GuardTime*, the node is considered de-synchronized, and leaves the gateway.

A BLE MTU of 255 B takes 1.02 ms to transmit at 2 Mbps. Accounting for the *TsTxOffset*, plus some *Padding* time needed by the TSCH state machine, the total duration of a **Mari** slot is 1.7 ms. Being significantly shorter than the 10 ms slots in IEEE 802.15.4e [17], this duration is key to keep system latency low (**R2**). **Mari** does not include acknowledgments at the link layer in order to keep the slot duration as short as possible, thus trading reliability for low latency.

The channel frequency used in a particular slot is computed as follows, based on [36]:

$$f = F[(ASN + channelOffset) \bmod n_{\text{freq}}], \quad (1)$$

where F is the list of available frequencies, ASN is the Absolute Slot Number, and n_{freq} is the number of frequencies. In **Mari**, the set of frequencies will depend on the type of slot, as defined in the next subsection.

3.5. Scheduling

Mari defines four types of slots: Beacon (B), Shared uplink (S), Downlink (D), and Uplink (U). Beacon slots are used to advertise the network: gateways emit **Beacon** frames, nodes listen for them. To speed up network discovery, Beacon slots hop through a limited set of frequencies, only using the 3 standard BLE advertising frequencies. Other slots hop through the remaining 37 BLE frequencies.

- ◆ Shared uplink slots are used by nodes to join a gateway and are subject to contention. They carry **JoinRequest** frames. Downlink slots are used by gateways to send **Data** frames. All nodes listen to Downlink slots, and they decide to accept or drop the frame based on the destination address. Finally, Uplink slots are unicast slots used by nodes to send data to a gateway.

A sequence of slots is called a slotframe that repeats over time. A set of slots with assigned slot types and sender/receiver pairs is called a schedule,

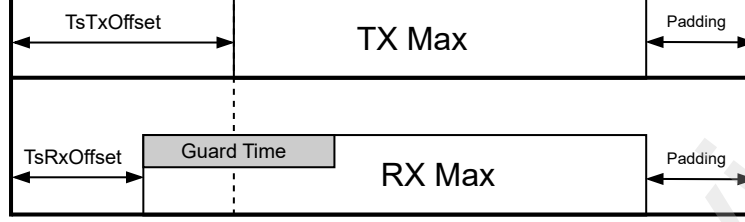


Figure 4: Chronogram inside a **Mari** slot. TX and RX Max are the maximum time that a transmitter or receiver will spend with the radio on. The receiver turns on $GuardTime/2$ μs before TX Max starts, enabling detection and correction of clock drifts. The total duration of a **Mari** slot is 1.7 ms.

Table 1: Summary of four selected schedules used in **Mari**. Slots are 1.7 ms long and carry frames with $MTU = 255$ B.

ID	Name	# of Slots	# Uplink	# Downlink	# Shared U.	Duration (ms)	Per-node Uplink		Per-gateway Downlink	
							kb/s	pkt/s	kb/s	pkt/s
1	huge	149	102	22	22	256.88	0.996	3.89	175.27	85.58
3	big	101	66	16	16	174.12	1.469	5.74	188.09	91.84
4	medium	67	44	10	10	115.51	2.217	8.66	177.36	86.60
6	tiny	17	10	2	2	29.31	8.735	34.12	139.76	68.24

which also repeats over time. In TSCH, a schedule is typically represented as a two-dimensional matrix of slots vs. communication channels. In wireless sensor network applications, that matrix is normally sparse so nodes can sleep during unused slots and conserve energy. Since **Mari** needs to have low latency and high scalability, all slots are used in every slotframe, therefore we represent the schedule simply as a unidimensional list of slots.

Mari adopts a semi-static scheduling approach. All gateways and nodes store a common set of pre-defined schedules. When a gateway starts, it selects one schedule to operate on, and broadcasts its identifier in the **Beacon** frame. Upon receiving a beacon, nodes can determine the active schedule and synchronize to the gateway. In each schedule, all slots types and positions are pre-defined, except for Uplink slots, which are dynamically allocated by the gateway as nodes join the network. Each node is assigned a dedicated Uplink slot that repeats every slotframe.

Fig. 2 contains an example schedule. It has 3 Beacon slots (one per advertising frequency), 2 Shared uplink slots, 2 Downlink slots, and 10 Uplink slots (this schedule supports up to 10 connected nodes). More Uplink slots can be added to increase scalability, but also increase latency. More Downlink slots allow a faster OTAP procedure, but impact other metrics. Similarly, more Shared uplink slots allow faster join times, but may impact latency and scalability. Next, we explore these tradeoffs and present a set of four schedules (see Tab. 1) that can be selected according to deployment-specific requirements of scale and latency.

3.5.1. Finding a Suitable Schedule

Scheduling in TSCH networks determines how slots are allocated between communicating peers. Typical TSCH schedules, designed for low-power sensor networks with infrequent traffic, offer high latency and limited throughput. Such designs do not meet the real-time needs of robotic swarms.

We explore a family of semi-static schedules that balances key trade-offs, including scalability (**R1**), low-latency (**R2**), throughput for OTAP updates (**R3**), as well as association and handover speed (**R4**, **R5**).

We create a discrete simulator that allowed us to tweak parameters and identify a set of four schedules that can be used in different contexts, with latency vs. scalability trade-offs. Table 1 presents the composition of each schedule along with key metrics.

The largest schedule (codename “huge”) has 149 slots and is capable of handling 102 robots per gateway (102 Uplink slots) with a total slotframe duration of 256.88 ms. Since each node only gets assigned a single Uplink slot, the slotframe duration directly impacts latency, e.g. it can be above 250 ms in case a node decides to send a frame just after its Uplink slot has passed. The “huge” schedule allows for almost 4 packets per second per node, representing a throughput close to 1 kB/s. The smallest schedule (codename “tiny”) is 29.31 ms long and can handle up to 10 nodes, offering an uplink throughput of 34 packets per second. In terms of downlink, schedules share a similar rate between 68 and

91 packets per second.

In addition to finding a suitable number of slots of each type, we also consider slot placement. Fig. 2 shows an example schedule. We aggregate the Beacon slots together to enable very fast discovery, even when each **Beacon** frame is sent in a different frequency. Remaining slots are evenly distributed to ensure relatively stable latency and association times. We place Downlink slots just after Shared uplink slots, allowing a gateway that receives a **JoinRequest** to immediately reply with a **JoinResponse** in the subsequent Downlink slot.

We also perform simulations to find optimal back-off parameters and adjust the number of Shared uplink slots. Simulation results for the “huge” schedule show that with $N_{\min} = 4$ and $N_{\max} = 6$, worst-case network formation time is less than 5 s. Since the ratio of Shared uplink slots is similar across all four schedules, the network formation time varies little across schedules, the main difference being the amount of nodes trying to concurrently join a single gateway. We use real nodes to validate these and other findings in Section 4.

3.6. Association

When a gateway turns on, it goes directly into **joined** state, meaning that it is fully operational: ready to emit frames in Beacon and Downlink slots, as well as to receive frames in Uplink and Shared uplink slots. When a node turns on, it needs to go through an association procedure before it can become operational. Fig. 5 illustrates the association procedure in **Mari**.

After initialization, a node is in **scanning** mode, where it listens for **Beacon** frames. Every *ScanWindow* ms, e.g. the duration of a slotframe, the node verifies if one or more beacons were received, and selects the one with the best RSSI. Gateways that are full (remaining capacity = 0) are excluded from this selection. The node learns the active schedule from the **Beacon** and synchronizes its clock to the gateway. It also learns the ASN, so that it can compute the frequency to use in a given slot. It is now at the **synced** state, ticking its schedule in sync with the gateway. The node needs to join the network using a Shared uplink slot, but since that slot is subject to contention, it first initializes

an exponential backoff state, defined as follows:

$$n \in [N_{\min}, N_{\max}], \quad W_n = 2^n - 1, \quad (2)$$

$$K \sim \mathcal{U}\{0, 1, \dots, W_n\}, \quad (3)$$

where n is the backoff state, W_n the contention window, K the number of Shared Uplink slots the node defers before retrying. On a collision, $n \leftarrow \min(n+1, N_{\max})$. On success, the backoff state resets. When the backoff gets to zero, the node moves into **joining** state and sends a **JoinRequest**. If no **JoinResponse** is received within a certain time, the node assumes a collision happened, and goes back into **synced** state with a new backoff state. If the node cannot join after some time, e.g. several failed attempts, it goes back into **scanning** state.

When the gateway receives a **JoinRequest**, it allocates an Uplink slot to that node, and communicates it back as part of a **JoinResponse**. The node then moves into **joined** state and becomes fully operational: it can now send frames in its assigned Uplink slot, and receive frames in Downlink and Beacon slots.

3.7. Disassociation

Disassociation events can occur due to timeouts, de-synchronization, and Bloom filter membership tests. Gateways refresh their timeout whenever they receive an uplink message. If a node is not seen for a *NodeTimeout* number of slotframes, it is removed from the list of joined nodes. Similarly, if the node does not hear beacons from the gateway for *GatewayTimeout* slotframes, it goes back to **scanning** state. As previously described, if a node detects it has drifted too much from the gateway, it considers itself de-synchronized and leaves the gateway.

One final issue is link asymmetry: if node and gateway use different transmission power settings, it may happen that the node still hears the beacons, while the gateway has already dropped the node. Mari solves it by efficiently encoding the list of joined nodes in the **Beacon** frame using a Bloom filter. A

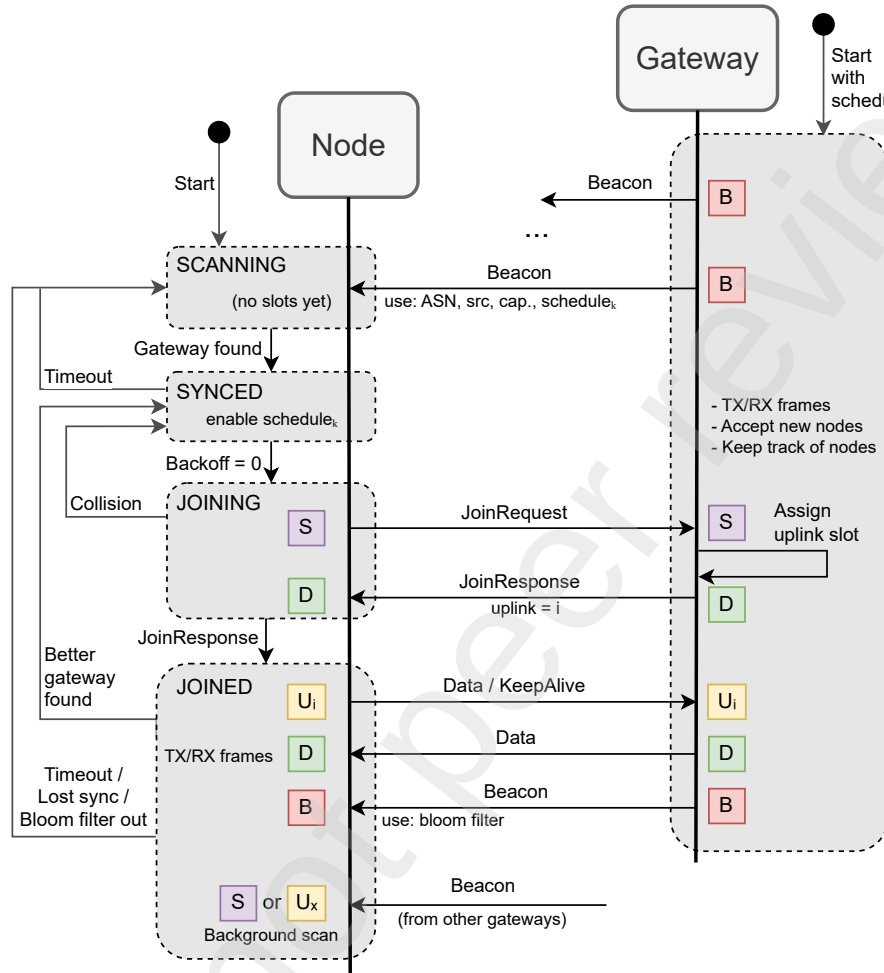


Figure 5: **Mari** state machine covering association, disassociation, regular operation, and handover capabilities.

Bloom filter [37] is a compact probabilistic data structure that encodes whether an element is a member of a set, trading extremely low memory use for a small probability of false positives. For example, we computed the probability of false positives for 100 nodes encoded in a 1024 bit Bloom filter to be less than 5%. False positives are not an issue, since the node will simply stay connected to the gateway a little bit longer.

The Bloom filter consists of a bit array and two hash functions. When a

node joins, the gateway hashes its address twice and sets the corresponding bits to 1. When a node receives a beacon, it applies the same hash functions to its own address and checks if all those bits are set. If they are, the node assumes it is still part of the network; if not, it detects that the gateway has dropped it and re-initiates the association procedure. This approach allows efficient membership checks even for one hundred nodes in a single gateway.

3.8. Handover

As we show in Section 3.5.1, the largest schedule in **Mari** supports up to 102 nodes while having an acceptable latency. While that scale already surpasses most of the current literature for low-power robots, it is still not enough to connect future swarms of thousands of robots. In particular, the DotBot testbed aims to connect 1,000 robots. We therefore adopt a horizontal scalability approach, where, to support more nodes, we simply add more gateways to the system. This implies the need for a handover mechanism.

To enable node mobility with fast handovers, **Mari** nodes scan for beacons in the background. That is, while still in `joined` state, instead of having the radio sleep during unused slots, it turns on for receiving **Beacon** frames from other gateways. Specifically, a node will use the Uplink and Shared uplink slots not assigned to itself for background scanning. Since motors typically consume $10\times$ - $100\times$ more power than a BLE radio, continuous background listening has negligible impact on energy usage. Moreover, background scanning can be disabled when the robot stops, since no handover is needed when it is not moving.

After *BGScan* ms receiving beacons, the node selects the gateway with best RSSI, and, if the signal is significantly better, it disconnects from the current gateway to join the new one. This is shown in Fig. 5, where the node changes state from `joined` to `syncd`. The node uses a difference in RSSI to make the handover decision: $\Delta_{RSSI} = RSSI_{new} - RSSI_{current}$. A handover is triggered only if $\Delta_{RSSI} > H_{th}$, where H_{th} is a predefined hysteresis threshold. We empirically found $H_{th} = 24$ dB when using an nRF5340-DK device as a gateway and an nRF52840-Dongle as a node. From the old gateway's perspective, the

timeout *NodeTimeout* soon triggers (typically less than one second), leading to the disassociation of the node.

A worst-case scenario in this approach is when dozens or hundreds of robots move together, e.g. in a flocking behavior. In such cases, if the association time is too high, robots may appear unresponsive. On the other hand, since robots move at different positions, they are expected to observe different RSSI values, and therefore perform the handover at slightly different times, mitigating contention issues. We experimentally validate this assumption in Section 4.

The multi-gateway handover approach allows **Mari** to scale to several hundreds, potentially thousands of nodes. Since the nodes control all the handover decision, gateways do not need to coordinate about the assigned Uplink slots in the TSCH schedule, or even with the MQTT Broker, simplifying deployment (**R7**). When a gateway receives a message from the Broker, it simply checks the destination address, and if it is *not* for one of its nodes, it drops the message. Finally, unlike synchronized multi-gateway systems [2, 32], **Mari** gateways are entirely uncoordinated, offloading handover decision to nodes for improved scalability (**R1**).

4. Evaluation

This section presents results of the evaluation of **Mari** using real large-scale testbeds with up to 200 nodes. First, we describe our implementation and evaluation platforms. Then, we present our experiments and results regarding association time, handover, latency and packet delivery ratio. For each experiment, we also discuss the implications for robotic swarms.

Except otherwise noted, we use the “huge” schedule from Tab. 1, which supports up to 102 nodes per gateway. Similarly, we use the backoff parameters $N_{\min} = 4$ and $N_{\max} = 6$.

4.1. Implementation

We provide `mari-fw`³, an open-source implementation of the **Mari** link layer for both nodes and gateways. `mari-fw` runs on the Nordic nRF5340 chip and on chips of the nRF52 family, such as the nRF52840. It is written in bare-metal C code following modern software engineering practices, including automated verification for code formatting. We also provide `marilib`⁴, a Python tool that can run in a computer or Raspberry Pi. It allows for both bridging an nRF gateway to an IP network via a UART interface, and for providing a dashboard for monitoring network status. It offers users an easy to use API to communicate with **Mari**-enabled nodes, and maintains a local state of the connected nodes. It also offers built-in metric computation in real time, including RSSI, PDR and latency. By providing a fully open-source implementation that runs on COTS hardware, we not only ensure that all our experiments can be replicated, but also that **Mari** can be used as a platform for experimentation in large-scale robotic swarms (**R7**).

4.2. Evaluation Platforms

Our evaluation relies on three hardware platforms, as shown in Fig. 6 and described below.

DotBot. DotBot is a custom-built two-wheeled mobile robot that integrates motors, a transceiver, and onboard control electronics. It uses the nRF5340 MCU, and serves as the mobile platform in our experiments [38, 35].

Mobile Panels. Consists of a set of ten custom built 3D-printed panels, each hosting ten nRF52840 Dongle nodes, resulting in 100 **Mari** nodes in total. Each panel has an on/off switch, offering a portable and scalable way to deploy and test large numbers of nodes. The panels can be easily moved around for testing handover behavior.

³<https://github.com/dotbots/mari>

⁴<https://github.com/dotbots/marilib>

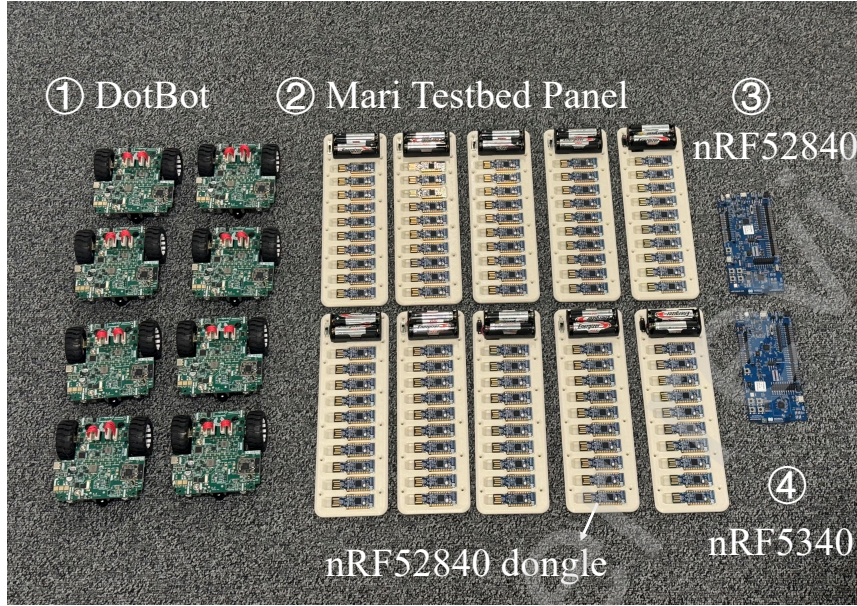


Figure 6: The hardware platforms used in our evaluation.

nRF52840-DK. A development kit based on the Nordic nRF52840 SoC, which we use as a node running **Mari**. We have access to a wall-mounted testbed which includes 50 nRF52840-DKs.

nRF5340-DK. Also a development kit, based on the dual-core nRF5340 chip. We use it for both running **Mari** gateways and nodes. When used as gateway, we use standalone boards connected to a computer via a USB cable. When used as node, we use it in the context of the aforementioned wall-mounted testbed, which also contains 50 nRF5340-DKs.

4.3. Association Time

We evaluate the network formation time under the worst case in which all nodes are already powered on and a single gateway is subsequently activated. The goal is to evaluate association time under severe contention. In the setup, we use the Mobile Panels as the experimental platform. We vary the number of nodes from 20 to 100 in steps of 20 and repeat each experiment ten times.

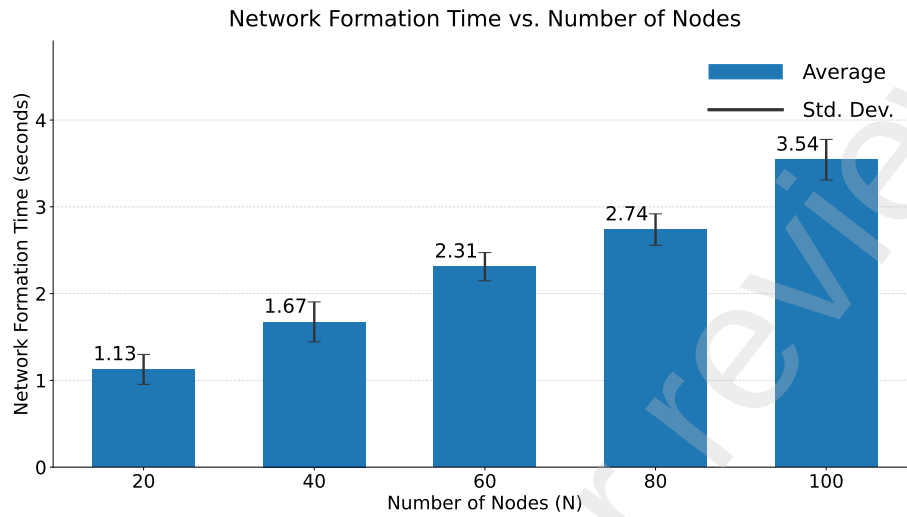


Figure 7: Worst-case network formation time as a function of the number of nodes. We measure the case of a “join storm”, when all nodes discover a single gateway at once.

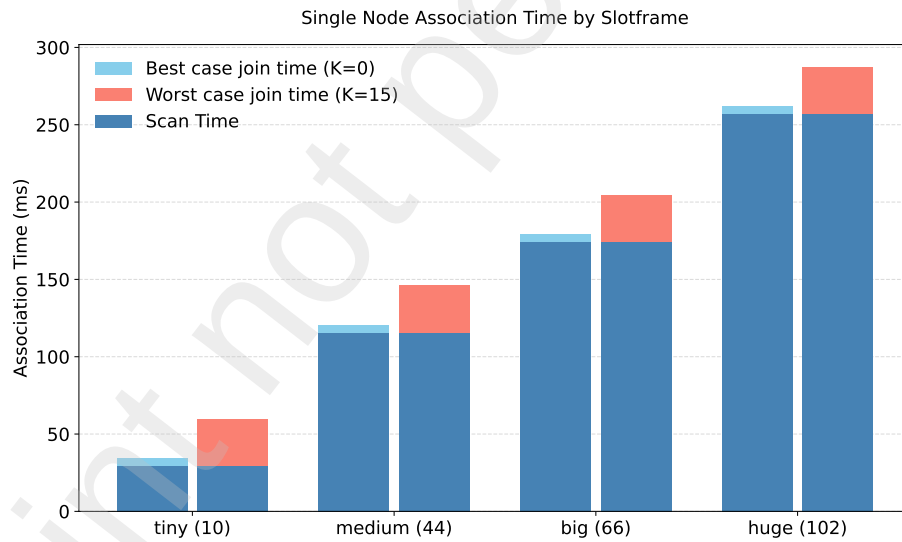


Figure 8: Best and worst case association time for a single node, when no contention is observed.

For each run, we record the duration between the moment when no nodes are connected and the time when the last node successfully joins the network.

As shown in Fig. 7, the formation time increases with network size. For networks of 20 and 40 nodes, the process completes within 2 s, while for networks of 100 nodes it takes close to 3.5 s. While the formation time is expected to be non-linear due to the exponential backoff (see Section 3.6), the selected parameters for $N_{\min}$ and $N_{\max}$ are optimized for large networks, therefore an exponential delay increase with number of nodes is not observed. The achieved results satisfy the requirement of fast association (**R4**), even at large scale (**R1**).

We also evaluate the association time of a single node without contention, i.e. no collisions experienced during network join, and present results in Fig. 8. Since `mari-fw` is configured to stay in `scanning` state for a full slotframe length, the scan time for a “huge” slotframe is 256.88 ms. This gives the node a chance to receive at least one beacon from each gateway in range. While the scan time depends on the slotframe duration, the join time (without contention) depends on the initial backoff parameter K , and is the same for all slotframes. Once a node is synchronized, in the best case where $K = 0$, it takes 6.8 ms (4 slots) for it to receive a `JoinResponse`. When $K = 2^4 - 1 = 15$, 32.3 ms (18 slots) are needed before the node can join. Therefore, the best and worst association time for a single node using a “huge” slotframe, from beginning of the scan window to fully joined, are 263.68 ms and 289.18 ms, respectively. When using the “tiny” slotframe, these numbers reduce to 36.11 ms in the best case and 61.61 ms in the worst case.

These results also have implications for the handover time. During handover, the scanning phase is omitted, as nodes continuously listen for beacons in the background. Consequently, the dominant factors become the join duration and the backoff. Therefore, in the best case, even when using the “huge” slotframe, a node can fully perform a handover in less than 50 ms. Next, we investigate the handover duration in large scale deployments.

4.4. Handover Time

We evaluate how long it takes for a large group of nodes to handover from one gateway to another. Our goal is to quantify the downtime caused during worst-

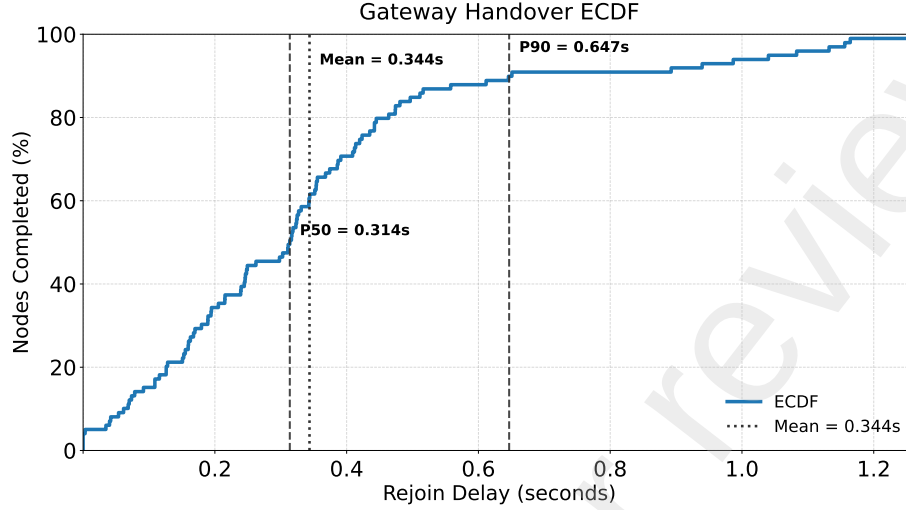


Figure 9: Gateway handover duration across 100 nodes.

case handover events, where re-association potentially causes severe contention.

We use 100 nodes and two gateways, where we use ten Mobile Panels and two nRF5340-DK. Each gateway is connected to a separate computer running `marilib`. With all nodes initially connected to a gateway, we physically move the 10 panels at walking speed so that all nodes reconnect to the other gateway.

Fig. 9 shows the Empirical Cumulative Distribution Function (ECDF) of handover delays for 100 nodes, measured from each node leaving one gateway to its first join on the other. Half of the nodes rejoin in under 0.314 s ($P50 = 0.314$ s), and 90% rejoin in under 0.647 s. The average rejoin delay is 0.34 s, with more than 80% of the nodes completing the handover in less than half a second. All 100 nodes complete handover within 1.3 s. Note that this time is much smaller than the 3.4 s obtained for 100 nodes in Fig. 7 because, during handover, the moving nodes have slightly different RSSI readings, causing handover attempts to happen at slightly different times, thereby reducing the number of collisions in Shared uplink slots. This demonstrates that the approach of multiple non-coordinated gateways used in Mari allows for dozens of robots to move between gateways with small downtime impacts, satisfying

requirements **R5** and **R1**.

4.5. Latency

We measure the round-trip latency in order to quantify the response time of a given robot, under different scales. We measure end-to-end latency using a request/response probe packet, where the corresponding reply is used to compute the round-trip time (RTT). To emulate realistic experimental conditions, we introduce background traffic corresponding to 20% schedule occupancy of Downlink slots, which we use as a representative load in our evaluation.

First, we evaluate latency using a single gateway, and vary the schedule. We consider the four schedules shown in Tab. 1, supporting up to 10, 44, 66, and 102 nodes. In each configuration, the network is populated to the schedule's full capacity and run for 5 minutes. Fig. 10 shows the average per-node RTT across the four schedules, with error bars indicating the standard deviation. The results indicate that latency increases with the schedule size, from an average of 45.7 ms in the “tiny” schedule to 269.3 ms in the “huge” schedule. These align with the schedules parameters in Tab. 1: the measured latency closely follows the slotframe duration, with only a small delay introduced by the 20% Downlink occupancy used during experiments. A noticeable standard deviation appears in our measurements. This is because probes are generated periodically, while transmission opportunities occur only at discrete cells; thus, a probe sent just after an Uplink slot may wait nearly a full slotframe, whereas one sent just before an Uplink slot experiences little delay.

We also evaluate a scenario with two gateways, each configured with the “huge” schedule. Both gateways were connected to a single computer running local MQTT broker. Together, they serve a total of 200 nodes. We determine the average latency to be 274 ms. As expected, the latency remains similar to the latency measured at the setup with a single gateway. This demonstrates the horizontal scalability of **Mari** (**R1**), where adding more gateways has no significant impact on responsiveness (**R2**).

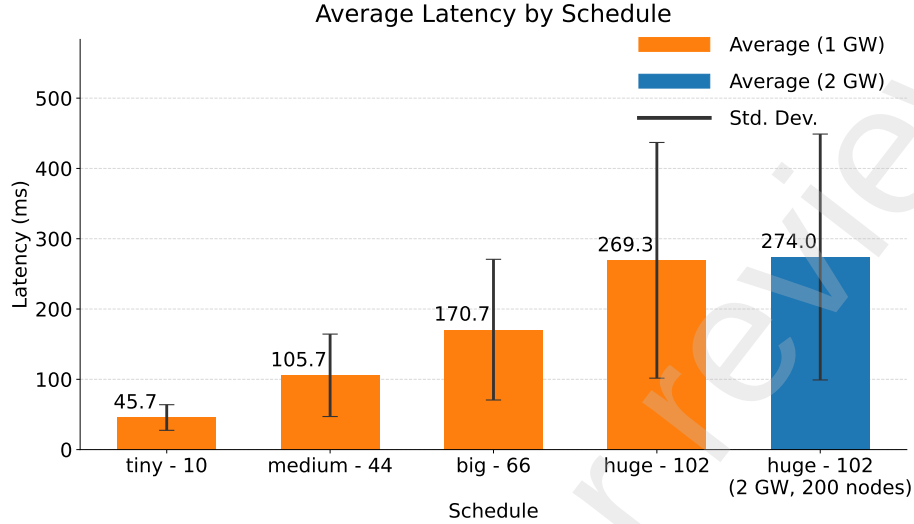


Figure 10: Latency evaluation under different schedule sizes. Bars report the average RTT per node, while error bars show the standard deviation.

4.6. Packet Delivery Ratio

To quantify communication reliability beyond latency, we evaluate the PDR, which is also critical for multi-robot networks. We use two setups: one with 100 nodes connected to a single gateway, another with 200 nodes served by two gateways. All nodes are placed in close proximity, approximately 2 m away from gateways. Similarly, gateways are less than 1 m apart from each other. We conduct experiments in a non-controlled RF environment, with several Wi-Fi networks in range. We emulate the same 20% schedule occupancy as in the latency experiment and measure PDR with one gateway connected to 100 nodes, and then with two gateways serving 200 nodes. Each experiment runs for 2 min, and we compute a sliding-window PDR, separating uplink and downlink communications. Note that there are no link-layer acknowledgments in *Mari*, so the PDR is measured for a single transmission attempt.

Table 2 shows the results, with the 100-node setup achieving PDR of 97.7% for uplink and 98.44% for downlink. This indicates that even when a gateway is serving a large number of nodes, network operation remains reliable. The

Table 2: Average PDR for uplink and downlink with different numbers of nodes and gateways.

Nodes	Gateway(s)	Direction	Average PDR (%)
100	1	Uplink	97.70 ± 0.53
		Downlink	98.44 ± 0.64
200	2	Uplink	93.30 ± 1.07
		Downlink	95.14 ± 0.94

200 node setup with two gateways achieves a slightly lower PDR of 93.3% and 95.14% for uplink and downlink, respectively. Although performance remains solid, this drop can be attributed to occasional inter-gateway interference, as de-synchronized schedules may overlap over time.

The interference between de-synchronized gateways can be bounded analytically. With a transmission duration of 1.02 ms per slot, two slots from different gateways collide only when they overlap in time and happen to use the same frequency. Since *Mari* hops across 37 BLE data channels, the probability of a frequency match is only $1/37$ ($\approx 2.7\%$), meaning that the vast majority of timing overlaps do not result in actual collisions. The 2-gateway PDR results in Tab. 2 confirm that performance remains above 93% even under worst-case proximity conditions (<1 m between gateways).

4.7. Memory Usage and Energy Consumption

To assess the impact on real robot platforms, we measure both memory usage and energy consumption. For memory, we consider a node configured with the huge schedule (max 102 nodes). In this configuration, an nRF52840 build requires 32.1 kB of flash memory and 20.4 kB of RAM (Table 3).

Next, we measure energy consumption of a single nRF52840 dongle running *Mari* under the 10-node schedule. We select this schedule to improve readability of results in Fig. 11, as the results are similar for all other schedules. During operation, we record both current and voltage, from which we compute the charge consumed during a single slotframe. As detailed in Fig. 11, within the

Table 3: Memory footprint and charge during one slotframe on nRF52840.

Category	Metric	Value
Memory	Flash Memory	32.1 kB
	RAM	20.4 kB
Energy	Slotframe Duration	30.32 ms
	Charge	0.2783 mC

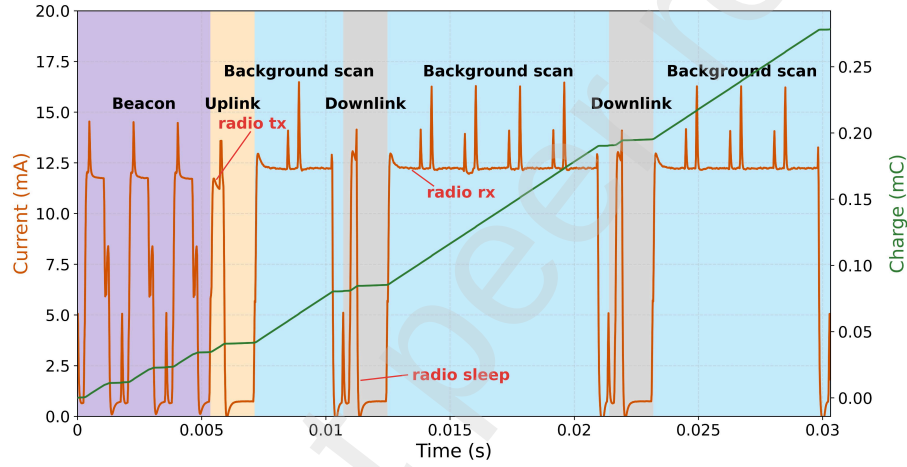


Figure 11: Current and voltage over one *tiny* slotframe (capacity: 10 nodes) on an nRF52840 dongle running **Mari**.

whole slotframe, the instrumented node uses the first Uplink slot. The remaining Uplink slots belong to other nodes, and the Shared uplink slots are not used in *joined* state; during these slots, the node performs background scanning. Over a full slotframe of approximately 30.32 ms, the node consumes 0.2783 mC of charge. This is negligible compared to the charge drawn by robot motors: at 300 mA [35], motors would draw 9.096 mC per slotframe, roughly 33× more than the radio.

5. Integration with the DotBot Testbed

This section presents preliminary results of integrating **Mari** with a large-scale swarm robotics testbed. As this integration is still ongoing, we report it separately from the main evaluation: we describe the integration process and key performance metrics, noting that these experiments were conducted in a less controlled environment.

As part of the OpenSwarm project, we built a testbed of 1,000 DotBots. The testbed, shown in Fig. 1, enables wireless instrumentation, remote firmware updates, and live remote control of low-power robots. Each robot uses an nRF5340 MCU [38] with a BLE transceiver and a 240 F supercapacitor-based energy system [35]. Under continuous operation, the supercapacitors can power the low-power radio (**R6**) for a few hours.

We modify the DotBot testbed software to use **Mari** as the communication layer. First, we replace the radio link layer of both the robot and the gateway with **mari-fw**. Next, we update the infrastructure to use **Mari**'s MQTT topics and modify the user software to use **marilib** as the communication adapter.

In an experimental deployment, we use 8 **Mari** gateways to connect up to 724 DotBots, as shown in Fig. 1. The **Mari** gateways are configured to use the *huge* schedule (max. 102 nodes), implying a maximum capacity of 816 nodes.

Users can flash firmware, start and stop experiments, and control robots remotely. We successfully flash a 4 kB firmware image in 4 min. To enable remote control, we integrate **marilib** with PyDotBot⁵, an environment for interacting with DotBot swarms that provides a web-based frontend for real-time monitoring of robot state, as well as a REST/Websocket interface for remote control. We develop Python scripts that interact with PyDotBot to command the entire swarm. We run an epidemic color-propagation experiment: a single robot is set to red, triggering its neighbors to turn red in turn, causing the color to spread across the swarm in real time. The propagation of the LED color and robot

⁵<https://github.com/DotBots/PyDotBot>

position used the **Mari** communication layer, meaning that all packets go all the way between the robot and the controlling script. We observe color changes in real time, as expected. This validate that **Mari** provides the low-latency, bidirectional connectivity needed for interactive swarm control.

This deployment allows us to validate several requirements in a practical, system-wide setting. The communication offers sufficient throughput for OTAP (**R3**), as well as the low latency needed for robot control (**R2**).

Finally, we aggregate logs from all gateways to compute network metrics. During a 30-min experiment window, the network achieves an average round-trip latency ≈ 270 ms and an average PDR above 85%.

To our knowledge, this is the largest interactive robotic testbed with low-power robots fully connected using only BLE radios and a TSCH-based MAC layer.

6. Discussion

Scalability. We thoroughly evaluate **Mari** in a setup with up to 200 nodes and two gateways, and demonstrate a large scale deployment with 724 nodes using eight gateways. We achieve linear increase in latency up to 100 nodes using a single gateway, and no significant latency increase when using more gateways to connect up to 724 nodes. We achieve high PDR even at the scale of hundreds of nodes, with an average PDR above 85% and an average end-to-end latency of 270 ms. In future work, we plan to investigate how gateway placement influences cross-interference, as gateway deployments of the same cardinality have different interference profiles depending on the gateway placement. The current version of **Mari** represents a significant advance over the state-of-the-art, where Wi-Fi based swarms connected 50 robots with 40 % packet loss [15] and BLE swarms needed 14 gateways to connect 70 robots [3].

Latency. We present a configurable scheduling approach that allows average latency between 45.7 ms and 269.3 ms for slotframes of capacity 10 and 102 nodes, respectively. These results are compatible with the literature on robot swarms.

The SHMG architecture reports latency figures between 300 and 500 ms [2]. Other systems report communication rates of “a few Hertz” [12] and control loops running every 0.27 s [15]. While CrazySwarm [14] reports latency as low as 26 ms for an average of 16.3 nodes per gateway, frame MTUs are only a few bytes long and the protocol only supports broadcast. Finally, since the **mari-fw** implementation is parameterized, it is possible to tune the MTU and other configurations to achieve more aggressive latency profiles.

Reliability. **Mari** intentionally omits link-layer acknowledgments to maximize throughput, leaving reliability to upper-layer protocols. This is an important design consideration: applications must implement their own mechanisms (e.g. retries and timeouts) to handle packet loss. The DotBot testbed integration illustrates this trade-off: OTAP initially failed for a portion of the 724-robot deployment due to insufficient retransmission timeouts at the application layer, an issue resolved by tuning these parameters (Section 5). As future work, **Mari** can be integrated with a standard network stack, where protocols such as TCP could provide reliable communication.

Security. **Mari** currently does not include a security layer, which is acceptable for experimental deployments in controlled environments, such as in robotic testbeds and research experiments. As a provisional measure, basic confidentiality and authentication can be handled at higher layers if needed. In future work, we plan to integrate secure join and frame protection mechanisms in **Mari**. This includes the use of standardized protocols for security in low-power networks, such as Ephemeral Diffie-Hellman Over COSE (EDHOC) [39] for key establishment, CBOR Object Signing and Encryption (COSE) [40] for message protection, and Lightweight Authorization over EDHOC (ELA) [41] for automated node enrollment. Furthermore, fast security resumption during handovers can be achieved via EDHOC-PSK [42].

Interoperability. Although **Mari** was developed and evaluated in a standalone fashion, in the future it could be integrated with a standard networking stack.

The first 6 bytes of identifiers defined in the frame format (Fig. 3) can be used as MAC addresses, and `marilib` could implement a TUN/TAP interface to expose `Mari` as a regular network interface as recognized by the operating system. This approach would impose higher complexity at the node side, as it would also have to manage more complex network stacks carried over `Mari` frames.

Scheduling. The use of semi-static scheduling brings a balance between implementation simplicity, handover speed, and achieving communication requirements (**R1-R5**). In the future, instead of specifying a fixed set of schedules, a template could be used, and a protocol for dynamic schedule update could be specified to handle association and disassociation of new nodes. Impacts on relevant metrics should be considered, such as end-to-end latency, association time, and handover duration.

7. Conclusion

This paper introduces `Mari`, a platform demonstrated to connect more than 700 small robots while providing responsive communication and fast firmware updates. We use TSCH to enable predictable latency and mitigate interferences, and provide four semi-static schedules that balance several requirements, including latency, scalability, and association time. We perform extensive evaluation on real hardware, showing how `Mari` can associate 100 nodes in less than 4 s and achieve latency of 45.7 ms and 269.3 ms for 10 and 102 connected nodes, respectively. We simulate a flocking behavior with 100 real nodes, achieving an average handover time of only 0.34 s even under severe contention. We demonstrate 200 nodes connected to two gateways achieving latency of 274 ms, as well as an uplink and downlink PDR above 93% and 95%, respectively. Our solution is open-source and runs on COTS low-power micro-controllers. Besides the firmware, we provide libraries that allow users to plug-and-play `Mari` in their swarm deployments. We integrated `Mari` as the communication layer in the DotBot testbed, where we validated real time robot control and the firmware

update of 600 robots in less than 4 minutes. We plan to use **Mari** to connect the full testbed comprising 1,000 robots.

Acknowledgements

This document is issued within the frame and for the purpose of the OpenSwarm project. This project has received funding from the European Union’s Horizon Europe Framework Programme under Grant Agreement No. 101093046. Views and opinions expressed are however those of the author(s) only and the European Commission is not responsible for any use that may be made of the information it contains. We also thank Genki Miyauchi and Mohamed S. Talamali for contributing example DotBot applications, and William Takeshi Pereira, Amy McMahon, and Kate O’Riordan for their help deploying the DotBot testbed.

References

- [1] J. M. Kahn, R. H. Katz, K. S. Pister, Next Century Challenges: Mobile Networking for “Smart Dust”, in: ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom), Seattle, WA, USA, 1999.
- [2] J. Haxhibeqiri, A. Karaağaç, I. Moerman, J. Hoebeke, Seamless Roaming and Guaranteed Communication Using a Synchronized Single-hop Multi-Gateway 802.15.4e TSCH Network, *Ad Hoc Networks* 86 (2019) 1–14.
- [3] Nakagaki, Ken and Leong, Joanne and Tappa, Jordan L and Wilbert, João and Ishii, Hiroshi, Hermits: Dynamically Reconfiguring the Interactivity of Self-Propelled TUIs with Mechanical Shell Add-Ons, in: ACM Symposium on User Interface Software and Technology (UIST), New York, NY, USA, 2020.
- [4] L. Yang, J. Yu, S. Yang, B. Wang, B. J. Nelson, L. Zhang, A survey on swarm microrobotics, *IEEE Transactions on Robotics* 38 (3) (2021) 1531–1551.

- [5] J. Gielis, A. Shankar, A. Prorok, A Critical Review of Communications in Multi-Robot Systems, *Current Robotics Reports* 3 (4) (2022) 213–225.
- [6] G. Miyauchi, Y. K. Lopes, R. Groß, Sharing the Control of Robot Swarms Among Multiple Human Operators: A User Study, in: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2023, pp. 8847–8853.
- [7] M. Dorigo, G. Theraulaz, V. Trianni, Swarm Robotics: Past, Present, and Future [Point of View], *Proceedings of the IEEE* 109 (7) (2021) 1152–1165.
- [8] Le Goc, Mathieu and Kim, Lawrence H. and Parsaei, Ali and Fekete, Jean-Daniel and Dragicevic, Pierre and Follmer, Sean, Zooids: Building Blocks for Swarm User Interfaces, in: *ACM Symposium on User Interface Software and Technology (UIST)*, ACM, 2016, pp. 97–109.
- [9] Pickem, Daniel and Glotfelter, Paul and Wang, Li and Mote, Mark and Ames, Aaron and Feron, Eric and Egerstedt, Magnus, The Robotarium: A Remotely Accessible Swarm Robotics Research Testbed, in: *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2017, pp. 1699–1706.
- [10] Jeung, Matthew and Sathya, Anup and Qian, Wanli and Arellano, Steven and Jimenez, Luke and Nakagaki, Ken, Shape n’Swarm: Hands-on, Shape-Aware Generative Authoring with Swarm UI and LLMs, in: *ACM Symposium on User Interface Software and Technology (UIST)*, ACM, 2025, pp. 1–16.
- [11] Retnanto, Aditya and Faracci, Emilie and Sathya, Anup and Hung, Yu-Kai and Nakagaki, Ken, CARDinality: Interactive Card-Shaped Robots with Locomotion and Haptics Using Vibration, in: *ACM Symposium on User Interface Software and Technology (UIST)*, ACM, 2024, pp. 1–14.
- [12] Pichierri, Lorenzo and Testa, Andrea and Notarstefano, Giuseppe, Crazy-

- choir: Flying Swarms of Crazyflie Quadrotors in ROS 2, *IEEE Robotics and Automation Letters* 8 (8) (2023) 4713–4720.
- [13] J. Xu, T. Vu, D. S. D’Antonio, D. Saldaña, MochiSwarm: A Testbed for Robotic Micro-Blimps in Realistic Environments, in: *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 584–590.
- [14] Preiss, James A. and Honig, Wolfgang and Sukhatme, Gaurav S. and Ayanian, Nora, Crazyswarm: A Large Nano-Quadcopter Swarm, in: *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2017, pp. 3299–3304.
- [15] C. L. Liu, I. L. D. Ridgley, M. L. Elwin, M. Rubenstein, R. A. Freeman, K. M. Lynch, Self-Healing Distributed Swarm Formation Control Using Image Moments, *IEEE Robotics and Automation Letters* 9 (7) (2024) 6216–6223.
- [16] Alhafnawi, Merihan and Gomez-Gutierrez, Maca and Hunt, Edmund R. and Lemaignan, Severin and O’Dowd, Paul and Hauert, Sabine, Express Yourself: Enabling Large-Scale Public Events Involving Multi-Human-Swarm Interaction for Social Applications with MOSAIX, in: *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 577–583.
- [17] IEEE Standards Association, IEEE Standard for Local and Metropolitan Area Networks—Part 15.4: Low-Rate Wireless Personal Area Networks (LR-WPANs) Amendment 1: MAC Sublayer, *IEEE Std 802.15.4e-2012*, accessed: 2026-02-11 (Apr. 2012). doi:10.1109/IEEESTD.2012.6185525. URL <https://doi.org/10.1109/IEEESTD.2012.6185525>
- [18] D. Dujovne, T. Watteyne, X. Vilajosana, P. Thubert, 6TiSCH: Deterministic IP-Enabled Industrial Internet (of Things), *IEEE Communications Magazine* 52 (12) (2014) 36–41.

- [19] Wilson, Sean and Egerstedt, Magnus, The Robotarium: A Remotely-Accessible, Multi-Robot Testbed for Control Research and Education, IEEE Open Journal of Control Systems 2 (2022) 12–23.
- [20] M. Baert, J. Rossey, A. Shahid, J. Hoebeke, The Bluetooth Mesh Standard: An Overview and Experimental Evaluation, Sensors 18 (8) (2018) 2409.
- [21] Ichihashi, Sosuke and Kuroki, So and Nishimura, Mai and Kasaura, Kazumi and Hiraki, Takefumi and Tanaka, Kazutoshi and Yoshida, Shigeo, Swarm Body: Embodied Swarm Robots, in: CHI Conference on Human Factors in Computing Systems (CHI), ACM, 2024, pp. 1–19.
- [22] Rubenstein, Michael and Ahler, Christian and Nagpal, Radhika, Kilobot: A low cost scalable robot system for collective behaviors, in: IEEE International Conference on Robotics and Automation (ICRA), Saint Paul, MN, USA, 2012.
- [23] Ergen, Sinem Coleri, ZigBee/IEEE 802.15.4 Summary, Technical Report UCB/EECS-2004-17, University of California, Berkeley (September 2004). URL <https://pages.cs.wisc.edu/~suman/courses/707/papers/zigbee.pdf>
- [24] Thread Group, Thread Network Fundamentals (Version 3), White Paper 4001_1, Thread Group (May 2020).
- [25] S. Duquennoy, B. Al Nahas, O. Landsiedel, T. Watteyne, Orchestra: Robust mesh networks through autonomously scheduled tsch, in: Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems (SenSys), ACM, Seoul, South Korea, 2015, pp. 337–350. doi:10.1145/2809695.2809714. URL <https://doi.org/10.1145/2809695.2809714>
- [26] Watteyne, Thomas and Doherty, Lance and Simon, Jonathan and Pister, Kris, Technical Overview of SmartMesh IP, in: International Conference on

Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS), IEEE, 2013, pp. 547–551.

- [27] L. Harms, O. Landsiedel, TSCH Meets BLE: Routed Mesh Communication Over BLE, in: IEEE International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), IEEE, Pafos, Cyprus, 2023, pp. 187–195.
- [28] F. Graf, T. Watteyne, M. Villnow, What to Expect when Using DECT NR+, IEEE Communications Magazine (2025).
- [29] O. Nielsen, L. K. Schnügger, C. Orfanidis, X. Fafoutis, Mobility-Focused Joining in TSCH Networks, in: IEEE Conference on Local Computer Networks (LCN), IEEE, Edmonton, AB, Canada, 2022, pp. 271–274.
- [30] W. Jerbi, O. Cheikhrouhou, A. Guermazi, H. Trabelsi, MSU-TSCH: A Mobile Scheduling Updated Algorithm for TSCH in the Internet of Things, IEEE Transactions on Industrial Informatics 19 (7) (2022) 7978–7985.
- [31] W. Jerbi, O. Cheikhrouhou, A. Guermazi, H. Trabelsi, An Enhanced MSU-TSCH Scheduling Algorithm for Industrial Wireless Sensor Networks, Concurrency and Computation: Practice and Experience 36 (5) (2024) e7938.
- [32] M. Pettorali, F. Righetti, C. Vallati, S. K. Das, G. Anastasi, LASA-R: Location-Aware Scheduling Algorithm with Rescheduling for Industrial IoT Networks with Mobile Nodes, IEEE Internet of Things Journal 11 (9) (2024) 15735–15749.
- [33] M. Pettorali, F. Righetti, C. Vallati, S. K. Das, G. Anastasi, Mobility Management in Industrial IoT Environments, in: IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoW-MoM), IEEE, 2022, pp. 271–280.
- [34] S. Ben Yaala, F. Théoleyre, R. Bouallegue, Cooperative Resynchronization to Improve the Reliability of Colocated IEEE 802.15.4-TSCH Networks in Dense Deployments, Ad Hoc Networks 64 (2017) 112–126.

- [35] Liu, Mengyao and Deferme, Lowie and Van Eyck, Tom and Yang, Fan and Michiels, Sam and Abadie, Alexandre and Alvarado-Marin, Said and Maksimovic, Filip and Miyauchi, Genki and Jayakumar, Jessica and others, CapBot: Enabling Battery-Free Swarm Robotics, in: IEEE International Conference on Robotics and Automation (ICRA), IEEE, (Atlanta, GA, USA), 2025, pp. 570–576.
- [36] Watteyne, Thomas and Palattella, Maria Rita and Grieco, Luigi Alfredo, Using IEEE 802.15.4e Time-Slotted Channel Hopping (TSCH) in the Internet of Things (IoT): Problem Statement, Tech. Rep. RFC 7554, RFC Editor (May 2015). doi:10.17487/RFC7554.
URL <https://www.rfc-editor.org/info/rfc7554>
- [37] B. H. Bloom, Space/Time Trade-offs in Hash Coding with Allowable Errors, Communications of the ACM 13 (7) (1970) 422–426.
- [38] Alvarado-Marin, Said and Abadie, Alexandre and Maksimovic, Filip and Balbi, Martina and Savić, Trifun and Watteyne, Thomas, Demo: Dot-Bot, a cm-Scale, Easy-to-Use Micro-Robot for Swarm Research, in: Breaking Swarm Stereotypes Workshop at IEEE International Conference on Robotics and Automation (ICRA), IEEE, Yokohama, Japan, 2024.
- [39] G. Fedrecheski, M. Vučinić, T. Watteyne, Performance comparison of edhoc and dtls 1.3 in internet-of-things environments, in: Proceedings of the 2024 IEEE Wireless Communications and Networking Conference (WCNC), IEEE, 2024, pp. 1–6.
- [40] J. Schaad, CBOR Object Signing and Encryption (COSE): Structures and Process, Request for Comments (RFC) 9052, 66 pages (August 2022). doi:10.17487/RFC9052.
URL <https://www.rfc-editor.org/info/rfc9052>
- [41] G. Fedrecheski, G. Selander, T. Watteyne, M. Vučinić, Ela: Secure, lightweight, and zero-touch enrollment for iot devices, in: Proceedings of

the 21st Annual International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), 2025.

- [42] E. L. Perez, T. Watteyne, M. Vučinić, Pre-shared key authentication in ephemeral diffie-hellman over cose, in: Proceedings of the 11th International Conference on Internet of Things: Systems, Management and Security (IOTSMS), IEEE, 2024, pp. 254–257.